NSI S3 : Langage et programmation : Python 3

Après avoir vu comment fonctionne un ordinateur, nous allons programmer à l'aide de Python 3. Pour cela nous allons utiliser une interface où certaines bibliothèques sont déjà installées : Edupython : https://edupython.tuxfamily.org/.
Sur vos clés données en début d'année, vous avez une version portable (pas besoin de l'installer).

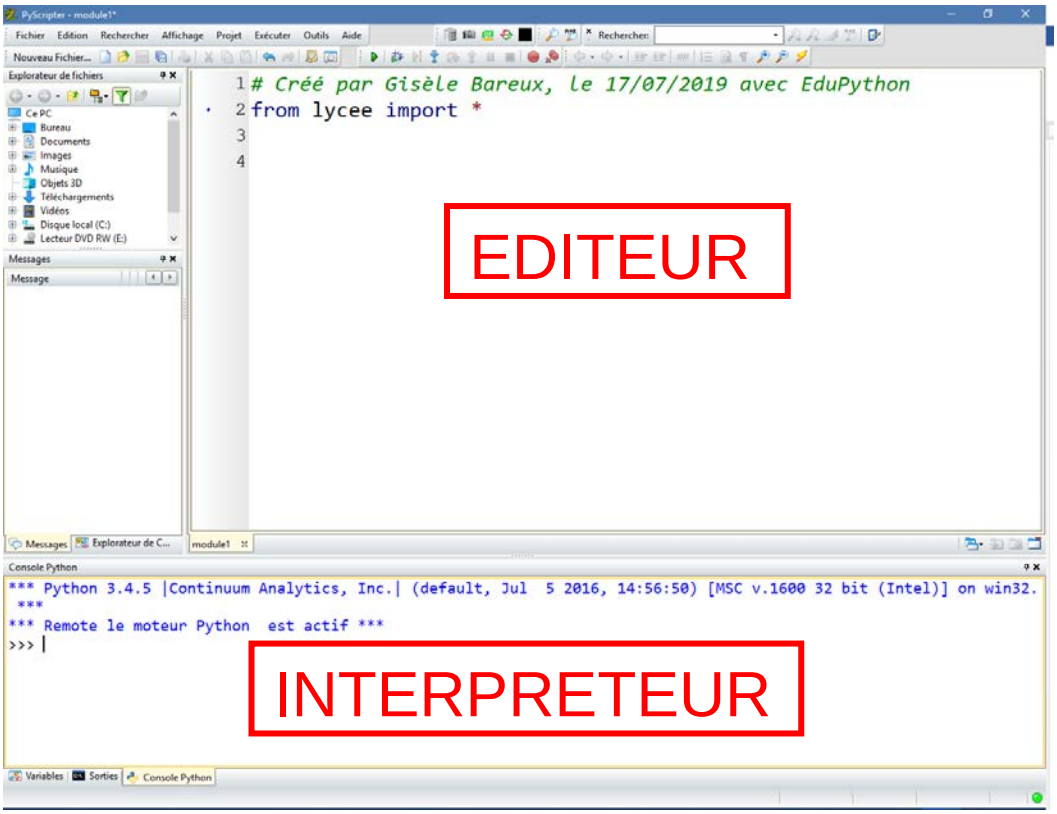
1. Histoire de Python :

En 1989, Guido van Rossum, profitant d'une semaine de vacances durant les fêtes de Noël, utilise son ordinateur personnel pour écrire la première version du langage. Fan de la série télévisée Monty Python's Flying Circus, il décide de baptiser ce projet Python.
Python est un langage multiplateforme, c'est-à-dire disponible sur plusieurs architectures (compatible PC, tablettes, smartphones, Raspberry Pi...) et systèmes d'exploitation (Windows, Linux, Mac, Android...).Le langage Python est gratuit, sous licence libre.

2. Prise en main du logiciel



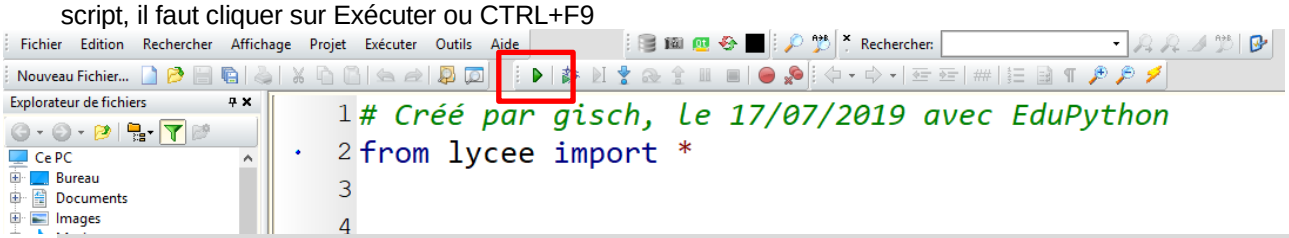
I) Exécuter EduPython.exe.
II) Puis faire Fichier/Nouveau Fichier et choisir Lycée et Créer. Vous arrivez sur l'interface suivant :



L'interpréteur, on le reconnait facilement. C'est lui qui contient le triple chevron >>> qui est l'invite de Python et qui signifie que Python attend une commande. On peut taper des lignes de codes qui sont exécutées instantanément lorsqu'on valide par la touche Entrée.

III) Pour tester, taper print("Bonjour tout le monde") puis Entrée. Qu'obtenez-vous dans l'interpréteur?
IV) Pour tester, taper 32+5/2+1.5 puis Entrée. Qu'obtenez-vous dans l'interpréteur ?

Dans l'éditeur, on peut y écrire des scripts, c'est-à-dire des programmes petits ou grands. Pour exécuter le script, il faut cliquer sur Exécuter ou CTRL+F9



V) Pour tester, taper print ("bonjour ") dans la ligne 3 puis Entrée. Puis 3.5+8+10. Exécuter le script. Qu'obtenez-vous dans l'interpréteur ? Est-ce normal ?

3. Les variables :

Pour cette partie, nous testerons directement **dans l'interpréteur**.

a) Les noms de variables :

Le nom d'une variable s'écrit avec des lettres **(non accentuées)**, des chiffres ou bien le souligné « _ ».
Le nom d'une variable **ne doit pas commencer par un chiffre**.
En langage Python, l'usage est de ne pas utiliser de lettres majuscules pour nommer les variables (celles-ci sont réservées pour nommer les classes).
Exemple : prenom, mon_nom, personne3

b) Le type int (interger : nombre entiers)

Commençons par créer une variable « age » en lui donnant la valeur 21 (c'est ce que l'on appelle affectation ou assignation). Pour afficher dans la console la valeur de la variable, il faut utiliser la fonction **print(« nom de la variable1 à afficher »)**

VI) Coder « **age = 21** » puis valider par entrée
Coder « **print(age)** » puis valider par entrée
Faire une capture d'écran du résultat.

Lorsqu'on utilise une variable, on peut connaitre son type à l'aide de la fonction : **type(«nom_variable»)**

Coder « **print(type(age))** » puis valider par entrée. Faire une capture d'écran du résultat.

Lorsque vous créé des scripts, il faut les commenter. Sur Python, il faut mettre un « # » devant.

VII) Coder « **# ceci est un commentaire** » puis valider par entrée.
Coder «**age = age + 1 # en plus court : age += 1**» puis valider par entrée
Coder «**print(age)**» puis valider par entrée
Coder «**age = age - 3 # en plus court : age -= 3**» puis valider par entrée
Coder «**print(age)**» puis valider par entrée
Coder «**age = age*2 # en plus court : age *= 2**» puis valider par entrée
Coder «**print(age)**» puis valider par entrée.
Faire une capture d'écran du résultat.

Sur ce type de variables, on peut faire différentes opérations :

- L'addition avec le symbole « **+** ».
- La soustraction avec le symbole « - ».
- La multiplication avec le symbole « ***** ».
- La division avec le symbole « ***/*** ».
- La division entière avec le symbole « ***//*** » (le résultat est la partie entière de la division).
- Le modulo avec le symbole « **%** » (le résultat est le reste de la division).
- La puissance avec le symbole « ****** ».

VIII) Donner les lignes de codes pour répondre au cahier des charges suivant :

- Créer une variable a = 6*3 + 2
- Créer une variable b = 20
- Créer une variable c = a + 2*b
- Afficher dans la même ligne a, b et c (pour afficher plusieurs variables avec la fonction print, il suffit de les séparer par une « , » dans la parenthèse).
- Créer une variable angle = 450
- Créer une variable tour et calculer le nombre de tour entier qu'il y a dans angle (on utilisera //)
- Créer une variable reste et calculer l'angle restant (on utilisera %)
- Afficher dans la même ligne angle, tour et reste
- Créer une variable puissance qui vaut 2 à la puissance 20 et afficher le résultat et son type.
- Créer une variable racine2 qui sera la racine carrée de 2 soit 2 à la puissance 0.5 le résultat et son type.
- Faire une capture d'écran du résultat.

c) Le type float (nombres en virgule flottante)

Pour déclarer un float, il suffit de mettre le séparateur décimal qui est un point sous python.

IX) Coder « **nb = 17.0** » puis valider par entrée.
Coder «**print(nb)**» puis valider par entrée
Coder «**print(type(nb))**» puis valider par entrée
Coder «**d = 14/3** » puis valider par entrée
Coder «**print(type(d))**» puis valider par entrée. Faire une capture d'écran du résultat.

On peut aussi écrire des réels au format scientifique avec la lettre e.

X) Coder « **nb = 1.32645e3**» puis valider par entrée.
Coder «**print(nb)**» puis valider par entrée
Faire une capture d'écran du résultat.

On peut aussi utiliser des fonctions mathématiques existantes. Pour cela, il faut importer un module math **import math** par exemple.
Pour connaitre toutes les fonctions d'un module, il suffit d'utiliser la fonction **dir(« nom du module »)**.
Pour utiliser une fonction d'un module, il suffit d'utiliser **module.fonction(argument).**

XI) Coder «**print(math.pi) # donnée pi du module math (nombre pi)** » puis valider.
Coder «**print(math.sin(math.pi/4.0)) # fonction sin() du module math (sinus)** » puis valider.
Coder «**print(math.sqrt(2.0)) # fonction sqrt() du module math (racine carrée)** » puis valider.
Coder «**print(math.exp(-3.0)) # fonction exp() du module math (exponentielle)** » puis valider.
Coder «**print(math.log(math.e)) # fonction log() du module math (logarithme népérien)** » puis valider.
Faire une capture d'écran du résultat.

d) Le type str (string : chaîne de caractères)

Pour créer une variable str, il faut l'écrire entre apostrophes ''.

XII) Coder «**nom = 'Dupont' # entre apostrophes** » puis valider.
Coder «**print(nom)** » puis valider.
Coder «**print(type(nom))** » puis valider.
Coder «**prenom = "Pierre" # on peut aussi utiliser les guillemets**» puis valider.
Coder «**print(prenom)** » puis valider.
Coder «**print(nom, prenom) # ne pas oublier la virgule**» puis valider.
Faire une capture d'écran du résultat.

On peut concaténer deux chaînes à l'aide de l'opérateur **« + ».**
On peut connaitre la taille d'une chaîne à l'aide de la fonction **len(« nom de la chaîne »).**

XIII) Coder «**chaine = nom + prenom # concaténation 2 chaînes** » puis valider.
Coder «**print(chaine)** » puis valider.
Coder «**chaine = prenom + nom # concaténation 2 chaînes** » puis valider.
Coder «**print(chaine)** » puis valider.
Coder «**chaine = prenom + ' ' + nom**» puis valider.
Coder «**print(chaine)** » puis valider.
Coder «**chaine = chaine + ' 18 ans' # en plus court : chaine += ' 18 ans'**» puis valider.
Coder «**print(chaine)** » puis valider.
Coder «**print(len(chaine))** » puis valider.
Faire une capture d'écran du résultat.

Que faire lorsqu'on a une apostrophe dans une variable str ?

XIV) Coder «**chaine = 'Aujourd'hui**'» puis valider.
Coder «**chaine = 'Aujourd\'hui' # séquence d'échappement \'**» puis valider.
Coder «**print(chaine)** » puis valider.
Coder «**chaine = "Aujourd'hui"**» puis valider.
Coder «**print(chaine)** » puis valider.
Faire une capture d'écran du résultat.

La séquence d'échappement \n représente un saut ligne. On peut utiliser les triples guillemets (ou les triples apostrophes) pour encadrer une chaîne définie sur plusieurs lignes :

XV) Coder «**chaine = 'Première ligne\nDeuxième ligne'**» puis valider.
Coder «**print(chaine)** » puis valider.
Coder «**chaine = """Première ligne**» puis valider.
Deuxième ligne""" » puis valider.
Coder «**print(chaine)** » puis valider.
Faire une capture d'écran du résultat.

La fonction **input()** lance une invite de commande (en anglais : prompt) pour saisir une chaîne de caractères.

XVI) Coder « **chaine = input("Entrer un nombre réel : ")** » puis valider.
Coder « **print(chaine)** » puis valider.
Coder « **print(type(chaine))** » puis valider.

Coder « **nombre = float(chaine) # conversion de type pour pouvoir faire un calcul**» puis valider.
Coder « **print(nombre**2)** » puis valider.
Faire une capture d'écran du résultat.

e) Le type bool (booléen)

Cette variable peut avoir deux valeurs : **True** (vrai) ou **False** (faux)
Avec ces variables, on peut utiliser les opérateurs de comparaisons :

Opérateur	Signification	Remarques
<	strictement inférieur	
<=	inférieur ou égal	
>	strictement supérieur	
>=	supérieur ou égal	
==	égal	Attention : deux signes ==
!=	différent	

On peut aussi utiliser les opérateurs logiques **and, or** et **not**.

XVII) Coder « **variable = True** » puis valider.
Coder «**print(variable)** » puis valider.
Coder «**print(type(variable))** » puis valider.
Coder «**b = 10** » puis valider.
Coder «**print(b > 8)** » puis valider.
Coder «**print(b == 5)** » puis valider.
Coder «**print(b != 10)** » puis valider.
Coder «**print(0 <= b <= 20)** » puis valider.
Coder «**note = 13.0** » puis valider.
Coder «**mention_ab = note >= 12.0 and note < 14.0 # ou bien : mention_ab = 12.0 <= note < 14.0** » puis valider.
Coder «**print(mention_ab)** » puis valider.
Coder «**print(not mention_ab)** » puis valider.
Coder «**print(note == 20.0 or note == 0.0)** » puis valider.
Faire une capture d'écran du résultat.

On peut aussi utiliser l'opérateur **in** avec les chaînes de caractères.

XVIII) Coder « **chaine = 'Bonsoir'** » puis valider
Coder «**# la sous-chaîne 'soir' fait-elle partie de la chaîne 'Bonsoir' ?** » puis valider
Coder «**resultat = 'soir' in chaine**» puis valider
Coder «**print(resultat)** » puis valider
Coder «**print('b' in chaine)** » puis valider
Faire une capture d'écran du résultat.

f) Le type tuple (séquence)

Un tuple est une séquence.

XIX) Coder « **mon_tuple = (5, 8, 6, 9)**» puis valider
Coder «**print(type(mon_tuple))** » puis valider.

Dans le code ci-dessus, la variable "mon_tuple" référence un tuple, ce tuple est constitué des entiers 5, 8, 6 et 9.Chaque élément du tuple est indicé :

- le premier élément du tuple (l'entier 5) possède l'indice 0
- le deuxième élément du tuple (l'entier 8) possède l'indice 1
- le troisième élément du tuple (l'entier 6) possède l'indice 2
- le quatrième élément du tuple (l'entier 9) possède l'indice 3

Pour accéder à un élément du tuple, il faut utiliser la "notation entre crochets"

XX) Coder « **print(mon_tuple[2])**» puis valider
Coder « **print(mon_tuple[..])**» en remplaçant les .. par l'indice qu'il faut pour afficher 8 puis valider
Coder « **mon_tuple = ("le", "monde", "bonjour")**» puis valider
Coder « **print(mon_tuple[2], " ", mon_tuple[0], " ", mon_tuple[1] , "!")**» puis valider
Faire une capture d'écran du résultat XIX) et XX).

g) Le type list (liste)

Il n'est pas possible de modifier un tuple après sa création (on parle d'objet "immutable"), si vous essayez de modifier un tuple existant, l'interpréteur Python vous renverra une erreur.
Les listes sont, comme les tuples, des séquences, mais à la différence des tuples, elles sont modifiables (on parle d'objets "mutables").

Pour créer une liste, il existe différentes méthodes : une de ces méthodes ressemble beaucoup à la création d'un tuple :

XXI) Coder « **ma_liste = [5, 8, 6, 9]**» puis valider
Coder «**print(type(ma_liste))** » puis valider.
Coder « **print(ma_liste[2])**» puis valider
Coder « **ma_liste[2] = 15 #on remplace l'élément d'indice 2 par 15**» puis valider
Coder « **print(ma_liste)**» puis valider
Coder « **ma_liste.append(33) #on ajoute un élément 33 car la liste est mutable**» puis valider
Coder « **print(ma_liste)**» puis valider
Coder « **del ma_liste[1] #on supprime l'élément d'indice 1**» puis valider
Coder « **print(ma_liste)**» puis valider
Coder « **print(len(ma_liste)) #on affiche la taille de la liste**» puis valider
Coder « **print(ma_liste)**» puis valider
Faire une capture d'écran du résultat

On peut créer une liste à partir d'une chaîne de caractères c'est l'indexage et le slicing.

XXII) Coder « **chaine = 'Dupont Pierre'** » puis valider
Coder «**print(chaine[0]) # premier caractère (indice 0)** » puis valider
Coder «**print(chaine[1]) # deuxième caractère (indice 1)** » puis valider
Coder «**print(chaine[1:4]) # slicing**» puis valider
Coder «**print(chaine[2:]) # slicing**» puis valider
Coder «**print(chaine[-1]) # dernier caractère (indice -1)** » puis valider
Coder «**print(chaine[-6:]) # slicing**» puis valider
Faire une capture d'écran du résultat et compéter le tableau ci-dessous :

Chaîne	D	u	p	o	n	t		P	i	e	r	r	e
Indice >0	0	1											
Indice <0												-2	-1

On peut faire des listes de listes

XXIII) Coder « **m = [[1, 3, 4], [5 ,6 ,8], [2, 1, 3], [7, 8, 15]]**» puis valider
Nous obtenons ainsi quelque chose qui ressemble beaucoup à un "objet mathématique" très utilisé: une matrice
Il est évidemment possible d'utiliser les indices de position avec ces "listes de listes". Pour cela nous allons considérer notre liste de listes comme une matrice, c'est à dire en utilisant les notions de "ligne" et de "colonne". Dans la matrice ci-dessus :
En ce qui concerne les lignes :

- 1, 3, 4 constituent la première ligne
- 5, 6, 8 constituent la deuxième ligne
- 2, 1, 3 constituent la troisième ligne
- 7, 8, 15 constituent la quatrième ligne

En ce qui concerne les colonnes :

- 1, 5, 2, 7 constituent la première colonne
- 3, 6, 1, 8 constituent la deuxième colonne
- 4, 8, 3, 15 constituent la troisième colonne

Pour cibler un élément particulier de la matrice, on utilise la notation avec "doubles crochets" : m[ligne][colonne] (la première ligne et la première colonne ont pour indice 0).

XXIV) Coder « **print(m[1][2])**» puis valider
Faire une capture d'écran du résultat XXII) et XXIII).
On pourrait s'interroger sur l'intérêt d'utiliser un tuple puisque la liste permet plus de choses ! La réponse est simple : les opérations sur les tuples sont plus "rapides". Quand vous savez que votre liste ne sera pas modifiée, il est préférable d'utiliser un tuple à la place d'une liste.

h) Le type dict (dictionaire)

Un dictionnaire stocke des données sous la forme **clé ⇒ valeur.** Une clé est unique et n'est pas nécessairement un entier (comme c'est le cas de l'indice d'une liste)

XXV) Coder « **moyennes = {'math': 12.5, 'anglais': 15.8} # entre accolades**» puis valider
Coder « **print(type(moyennes))** » puis valider
Coder « **print(moyennes['anglais'])# entre crochets**» puis valider
Coder « **moyennes['anglais'] = 14.3 # nouvelle affectation**» puis valider
Coder « **print(moyennes)** » puis valider
Coder « **moyennes['sport'] = 11.0 # nouvelle entrée**» puis valider
Coder « **print(moyennes)** » puis valider
Faire une capture d'écran du résultat

i) Exercices :

XXVI) Le numéro de sécurité sociale est constitué de 13 chiffres auquel s'ajoute la clé de contrôle (2 chiffres).
Exemple : 1 89 11 26 108 268 91
La clé de contrôle est calculée par la formule : 97 - (numéro de sécurité sociale modulo 97)
Retrouver la clé de contrôle de votre numéro de sécurité sociale.
Vérifier avec votre numéro de sécurité social.
Quel est l'intérêt de la clé de contrôle ?

XXVII) Afficher la valeur numérique de $\sqrt{(4,6^3 - 15/16)}$
Comparer avec votre calculatrice.

XXVIII) A partir des deux variables prenom et nom demandées à l'utilisateur (fonction input), afficher les initiales (par exemple LM pour Léa Martin).

XXIX) L'identifiant d'accès au réseau du lycée est construit de la manière suivante : initiale du prénom puis les 6 premiers caractères du nom (le tout en minuscule).
Exemple : Alexandre Lecouturier → alecout
A partir des deux variables prenom et nom demandées à l'utilisateur, construire l'identifiant et l'afficher.
Pour mettre une chaine de caractères en minuscules, il faut utiliser la méthode lower() (chaine.lower())
Pour mettre une chaine de caractères en majuscules, il faut utiliser la méthode upper() (chaine.upper())

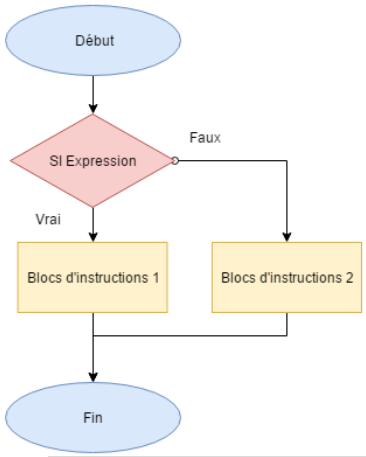
Pour aller plus loin :

XXX) On donne un extrait des logins d'accès au réseau du lycée :
- alecout Hwtrey
- lmartin montyo
- jloupin gnugpl

1) Créer une variable de type dict qui contient les couples identifiant - mot de passe ci-dessus.
2) La saisie du login fournit deux variables identifiant et mot_passe : une pour l'identifiant et l'autre pour le mot de passe.
La méthode keys() retourne la liste des clés du dictionnaire (dict.keys())
La méthode values() retourne la liste des valeurs du dictionnaire (dict.values())
Construire une variable booléenne qui donne True en cas d'identification correcte, et False dans le cas contraire :
On testera avec :
- lmartin montyo → True
- alecout hwtrey → False
- martin montyo → False
- lmartin gnugpl → False

4. Les conditions :

a) La condition IF:



La syntaxe de la condition est la suivante :

```
if (expression):          # ne pas oublier le signe de ponctuation ':'
    bloc d'instructions 1 # attention à l'indentation :(jaune)
else:                     # else est au même niveau que if
    bloc d'instructions 2 # attention à l'indentation
```

Si l'expression est vraie (True) alors le bloc d'instructions 1 est exécuté.
Si l'expression est fausse (False) alors c'est le bloc d'instructions 2 qui est exécuté.

XXXI) Copier le script ci-dessous, sauver sous exo31.py et tester le avec 33, 18 et 8 comme note:

```
chaine = input("Note sur 20 : ")
note = float(chaine)
if note >= 10.0:
    # ce bloc est exécuté si l'expression (note >= 10.0) est vraie
    print("J'ai la moyenne")
else:
    # ce bloc est exécuté si l'expression (note >= 10.0) est fausse
    print("C'est en dessous de la moyenne")
print("Fin du programme")
```

Faire une capture d'écran du résultat.

Pour traiter le cas des notes invalides (<0 ou >20), on peut imbriquer des instructions conditionnelles

XXXII) Copier le script ci-dessous, sauver sous exo32.py et tester le avec 33,20,0,18 et 8 comme note:

```
chaine = input("Note sur 20 : ")
note = float(chaine)
if note > 20.0 or note < 0.0:
    print("Note invalide !")
else:
    if note >= 10.0:
        print("J'ai la moyenne")
        if note == 20.0:
            # ce bloc est exécuté si l'expression (note == 20.0) est vraie
            print("C'est même excellent !")
    else:
        print("C'est en dessous de la moyenne")
        if note == 0.0:
            # ce bloc est exécuté si l'expression (note == 0.0) est vraie
            print("... lamentable !")
print("Fin du programme")
```

Faire une capture d'écran du résultat.

Une instruction elif (contraction de else if) est toujours associée à une instruction if.
Syntaxe :

```
if expression 1:
    bloc d'instructions 1
elif expression 2:
    bloc d'instructions 2
elif expression 3:
    bloc d'instructions 3      # ici deux instructions elif, mais il n'y a pas de limitation
else:
    bloc d'instructions 4
```

Si l'expression 1 est vraie alors le bloc d'instructions 1 est exécuté, et on passe à la suite du

programme.
Si l'expression 1 est fausse alors on teste l'expression 2 :

- si l'expression 2 est vraie on exécute le bloc d'instructions 2, et on passe à la suite du programme.
- si l'expression 2 est fausse alors on teste l'expression 3, etc.

Le bloc d'instructions 4 est donc exécuté si toutes les expressions sont fausses (c'est le bloc "par défaut").
Parfois il n'y a rien à faire. Dans ce cas, on peut omettre l'instruction else.

XXXIII) Copier le script ci-dessous, sauver sous exo33.py et tester le avec 33,20, 0, 18 et 8 comme note:

```
note = float(input("Note sur 20 : "))
if note == 0.0:
   print("C'est en dessous de la moyenne")
   print("... lamentable !")
elif note == 20.0:
   print("J'ai la moyenne")
   print("C'est même excellent !")
elif note < 10.0 and note > 0.0:      # ou bien : elif 0.0 < note < 10.0:
   print("C'est en dessous de la moyenne")
elif note >= 10.0 and note < 20.0:   # ou bien : elif 10.0 <= note < 20.0:
   print("J'ai la moyenne")
else:
   print("Note invalide !")
print("Fin du programme")
```

Faire une capture d'écran du résultat.

b) Exercices :

XXXIV) Vérification de la saisie d'un RIB, écrire le script qui répond au cahier des charges suivant :

- Demander le code de la banque et stocker l'entier dans une variable.
- Demander le code du guichet et stocker l'entier dans une variable.
- Demander le numéro de compte et stocker l'entier dans une variable.
- Demander la clé et stocker l'entier dans une variable.
- Stocker dans une variable de contrôle le calcul de la clé :
  Clé RIB = 97- ((89 x Code banque + 15 x Code guichet + 3 x Numéro de compte) modulo 97)
- Vérifier l'égalité entre la clé saisie et la clé calculée puis afficher à l'écran la validité ou non du RIB.

Sauver votre programme sous exo34.py puis tester votre programme avec le RIB suivant et faire une capture du résultat.

Banque	Guichet	N° compte	Clé	Devise
58496	**87451**	**00014500269**	**13**	**EUR**

XXXV) Ecrire le script qui calcule de l'IMC d'un adulte = Masse/taille² en kg/m² et qui l'interprète suivant le tableau ci-dessous :

IMC (kg·m−2)	Interprétation
moins de 16,5	dénutrition ou famine
16,5 à 18,5	maigreur
18,5 à 25	corpulence normale
25 à 30	surpoids
30 à 35	obésité modérée
35 à 40	obésité sévère
plus de 40	obésité morbide ou massive

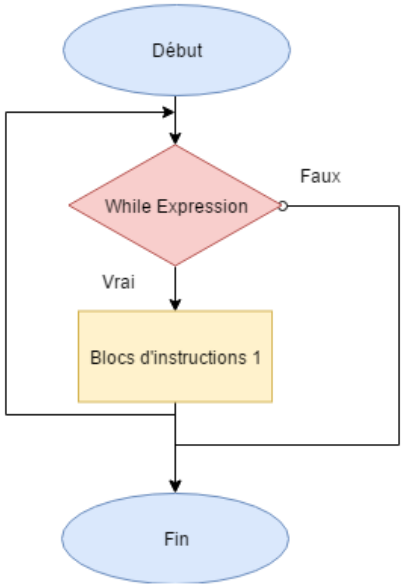
Le script demandera la taille en cm, la masse en kg puis affichera IMC = « Valeur de l'IMC » et Interprétation : « Colonne du tableau ci-dessus ».
Sauver votre programme sous exo35.py puis tester votre programme et faire une capture du résultat.

```
>>>
Votre taille en cm ? 170
Votre masse en kg ? 68.5
IMC =  23.7024221453 kg/m²
Interprétation : corpulence normale
>>>
```

5. Les boucles :

a) La boucle While :

Une boucle permet d'exécuter une portion de code plusieurs fois de suite.



La syntaxe de la boucle while est la suivante :

```
while expression:              # ne pas oublier le signe ':'
    bloc d'instructions 1      # attention à l'indentation.
```

Si l'expression est vraie (True) le bloc d'instructions 1 est exécuté, puis l'expression est à nouveau évaluée.
Le cycle continue jusqu'à ce que l'expression soit fausse (False).

XXXVI) Copier le script ci-dessous, sauver sous exo36.py :

```
# initialisation de la variable de comptage
compteur = 1
while compteur<=5:
    # ce bloc est exécuté tant que la condition (compteur<=5) est vraie
    print (compteur, "On est dans la boucle")
    compteur += 1    # incrémentation du compteur, compteur = compteur + 1
print ("On sort de la boucle")
```

Faire une capture d'écran du résultat.

XXXVII) Copier le script ci-dessous, sauver sous exo37.py :

```
# Table de multiplication par 5
compteur = 1# initialisation de la variable de comptage
while compteur <= 10:
    # ce bloc est exécuté tant que la condition (compteur <= 10) est vraie
    print(compteur, '* 5 =', compteur*5)
    compteur += 1    # incrémentation du compteur, compteur = compteur + 1
print("Et voilà !")
```

Faire une capture d'écran du résultat.

XXXVIII) Copier le script ci-dessous, sauver sous exo38.py :

```
# Affichage de l'heure courante
import time     # importation du module time
quitter = 'n'   # initialisation
while quitter != 'o':
    # ce bloc est exécuté tant que la condition est vraie
    # strftime() est une fonction du module time
    print('Heure courante ', time.strftime('%H:%M:%S'))
    quitter = input("Voulez-vous quitter le programme (o/n) ? ")
print("A bientôt")
```

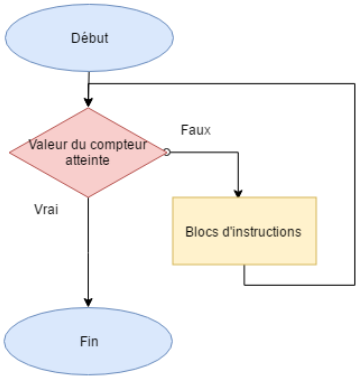
Faire une capture d'écran du résultat.

XXXIX) Copier le script ci-dessous, sauver sous exo39.py :

```
# Boucle infinie
i=0.0
while i != 1:
    i=i+0.1
    print(i)
print("A bientôt")
```

Faire une capture d'écran du résultat. Est-ce normal ?

b) La boucle FOR :



La syntaxe de la boucle For est la suivante :

```
for (élément) in (séquence):        # ne pas oublier le signe ':'
    bloc d'instructions             # attention à l'indentation.
```

XL) Copier le script ci-dessous, sauver sous exo40.py :

```
# Exemple avec une séquence de caractères
chaine = 'Bonjour'
for lettre in chaine:               # lettre est la variable d'itération
    print(lettre)
print("Fin de la boucle")
```

Faire une capture d'écran du résultat.

La variable lettre est initialisée avec le premier élément de la séquence ('B').
Le bloc d'instructions est alors exécuté.
Puis la variable lettre est mise à jour avec le second élément de la séquence ('o') et le bloc d'instructions à nouveau exécuté...
Le bloc d'instructions est exécuté une dernière fois lorsqu'on arrive au dernier élément de la séquence ('r').

XLI) Copier le script ci-dessous, sauver sous exo41.py :

```
# Exemple avec les éléments d'une liste
maliste = ['Pierre', 67.5, 18]
for element in maliste:
    print(element)
print("Fin de la boucle")
```

Faire une capture d'écran du résultat.

XLII)Copier le script ci-dessous, sauver sous exo42.py :

```
# Fonction range()
print(list(range(1,5)))
for i in range(1,5):
    print(i)
print("Fin de la boucle")
```

Faire une capture d'écran du résultat.

XLIII) Copier le script ci-dessous, sauver sous exo43.py :

```
# Table de multiplication
for compteur in range(1,11):
    print(compteur, '* 9 =', compteur*9)
print("Et voilà !")
```

Faire une capture d'écran du résultat.

c) L'instruction break

L'instruction break provoque une sortie immédiate d'une boucle while ou d'une boucle for.
Dans l'exemple suivant, l'expression True est toujours ... vraie : on a une boucle sans fin.
L'instruction break est donc le seul moyen de sortir de la boucle.

XLIV) Copier le script ci-dessous, sauver sous exo44.py :

```
import time     # importation du module time
while True:
   # strftime() est une fonction du module time
   print ('Heure courante ',time.strftime('%H:%M:%S'))
   quitter = input('Voulez-vous quitter le programme (o/n) ? ')
   if quitter == 'o':
      break
print ('A bientôt')
```

Faire une capture d'écran du résultat.

Si vous connaissez le nombre de boucles à effectuer, utiliser une boucle for.
Autrement, utiliser une boucle while (notamment pour faire des boucles sans fin).

d) Exercices :

XLV) Ecrire un script qui affiche toutes les tables de multiplication (de 1 à 10). Utilisez deux instructions "For" imbriquées avec deux variables différentes. Sauver votre programme sous exo45.py puis tester votre programme et faire une capture du résultat.

XLVI) Avec une boucle for, écrire un script qui compte le nombre de lettres o dans une chaîne de caractères.
Pour cela :
Pour chaque lettre de la chaine
  si la lettre est égale à 'o'
    alors incrémenter une variable
afficher la valeur de la variable
Sauver votre programme sous exo46.py puis tester votre programme et faire une capture du résultat.
A la suite du script, faire la même chose, directement avec la méthode count() de la classe str. (str.count('o'))
Sauver votre programme sous exo46.py puis tester votre programme et faire une capture du résultat.

XLVII)Ecrire le script du jeu de devinette d'un nombre entier aléatoire entre 1 et 100 suivant.
Pour créer un nombre entier aléatoire entre 1 et 100 :

```
import random
nombre = random.randint(1,100)
```

Sauver votre programme sous exo47.py puis tester votre programme et faire une capture du résultat.

```
--->   50
trop petit !
--->   75
trop petit !
--->   87
trop grand !
--->   81
trop petit !
--->   84
trop petit !
--->   85
Gagné en 6 coups !
```

XLVIII) Ecrire le script du code de César.
En cryptographie, le code de César est une technique de chiffrement élémentaire qui consiste à décaler une lettre de 3 rangs vers la droite :
A → D
B → E
...
Z → C
1) Ecrire le script de ce codage.

```
>>>
Message à coder ? abcdefghijklmnopqrstuvwxyz
defghijklmnopqrstuvwxyzabc
```

On pourra utiliser la chaîne 'abcdefghijklmnopqrstuvwxyz' et la méthode find() de la classe str.

```
>>> ch='abcdefghijklmnopqrstuvwxyz'
>>> a=str.find(ch,'z')
>>> print (a)
>>> 25
```

2) Ecrire le script de décodage.
Sauver votre programme sous exo48.py puis tester votre programme et faire une capture du résultat.